

PART XVI

MATHEMATICS AND NUMBER THEORY

The numeric toolkit competitive programming leans on
constantly

Several DP transitions in Part XV quietly assumed you could report an answer 'modulo 10^9+7 ' without explaining what that requires in practice. This Part supplies that missing machinery, and the rest of the standard competitive-math toolkit alongside it: primes and sieves, GCD/LCM, modular arithmetic and modular inverses, fast exponentiation, combinatorics and modular nCr , matrix exponentiation (revisiting Fibonacci one final time, now in $O(\log n)$), and the counting principles — inclusion-exclusion and the pigeonhole principle — that quietly underlie a large share of counting problems.

Contents of Part XVI

- 1. Divisibility, Factors, and Multiples
- 2. Primality Testing and Prime Factorization
- 3. Sieve of Eratosthenes and Segmented Sieve
- 4. GCD, LCM, and the Euclidean Algorithm
- 5. Extended Euclidean Algorithm and Modular Inverse
- 6. Modular Arithmetic
- 7. Binary (Fast) Exponentiation
- 8. Combinatorics: Permutations, Combinations, Factorials
- 9. Modular nCr and Pascal's Triangle
- 10. Matrix Exponentiation
- 11. Inclusion-Exclusion Principle
- 12. Pigeonhole Principle
- 13. Bitwise Mathematics Recap
- 14. Recognizing a Number-Theory Problem
- 15. Practice Drills
- Part XVI Revision, Master Cheat Sheet, and Interview Questions

Chapter 1 — Divisibility, Factors, and Multiples

1.1 The vocabulary

a divides b (written $a \mid b$) means $b = a * k$ for some integer k — equivalently, $b \% a == 0$. A factor (or divisor) of n is any number that divides n exactly; a multiple of n is any number n itself divides.

```
bool divides(long long a, long long b) { return b % a == 0; } // does
a divide b?
```

1.2 Finding all divisors of n in $O(\sqrt{n})$

Naively checking every i from 1 to n costs $O(n)$. The key observation: divisors come in pairs $(i, n/i)$, and one of every such pair is always $\leq \sqrt{n}$ — so it suffices to check i up to $\sqrt{n}$ and record both i and n/i whenever i divides n .

```
vector<long long> allDivisors(long long n) {
    vector<long long> divs;
    for (long long i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            divs.push_back(i);
            if (i != n / i) divs.push_back(n / i); // avoid
double-counting a perfect-square root
        }
    }
    return divs;
}
```

WARNING — Don't forget the $i \neq n/i$ check

When n is a perfect square, i and n/i coincide at $i = \sqrt{n}$ — pushing both would record that divisor twice. This exact loop-bound-plus-pairing shape ($i*i \leq n$) is identical to the primality check in Chapter 2 and the sieve's inner logic in Chapter 3 — the same $O(\sqrt{n})$ idea reused three times in this Part.

Approach	Complexity
Check every i from 1 to n	$O(n)$
Check i up to $\sqrt{n}$, pair with n/i	$O(\sqrt{n})$

Chapter 2 — Primality Testing and Prime Factorization

2.1 Checking primality in $O(\sqrt{n})$

```
bool isPrime(long long n) {  
    if (n < 2) return false;  
    for (long long i = 2; i * i <= n; i++)  
        if (n % i == 0) return false;  
    return true;  
}
```

The same pairing argument from Chapter 1 justifies this: if n has any factor greater than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$ — so if no factor exists up to $\sqrt{n}$, none exists at all.

2.2 Prime factorization in $O(\sqrt{n})$

```

vector<pair<long long,int>> primeFactorize(long long n) { // {prime,
exponent} pairs
    vector<pair<long long,int>> factors;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            int exp = 0;
            while (n % p == 0) { n /= p; exp++; }
            factors.push_back({p, exp});
        }
    }
    if (n > 1) factors.push_back({n, 1}); // one leftover prime
    factor larger than sqrt(original n)
    return factors;
}

```

Dry run: $n = 360$

p	n before	Divisions	n after	Recorded factor
2	360	$360/2=180$, $/2=90$, $/2=45$	45	(2, 3)
3	45	$45/3=15$, $/3=5$	5	(3, 2)
$p \cdot p > 5$ after $p=3$ ($p=4$ skipped as not dividing)	5	loop ends when $p \cdot p > 5$	5	leftover: (5,1)

$360 = 2^3 * 3^2 * 5^1$ — matches the well-known factorization.

NOTE — Why the leftover check matters

After the loop, n can have at most ONE remaining prime factor greater than $\sqrt{\text{original } n}$, because two such factors multiplied together would already exceed n . This is exactly why the leftover check only needs a single 'if $n > 1$ ', not another loop.

Chapter 3 — Sieve of Eratosthenes and Segmented Sieve

3.1 Why a sieve instead of repeated isPrime calls

Checking primality for every number up to N individually costs $O(N * \sqrt{N})$ total — too slow when N is large and many primality checks are needed. The Sieve of Eratosthenes instead marks all composites in one pass, achieving every check in $O(1)$ afterward, for $O(N \log \log N)$ total preprocessing.

```
vector<bool> sieveOfEratosthenes(int N) {
    vector<bool> isPrime(N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; (long long)p * p <= N; p++) {
        if (isPrime[p]) {
            for (int multiple = p * p; multiple <= N; multiple += p)
                isPrime[multiple] = false;
        }
    }
    return isPrime;
}
```

WARNING — Start marking multiples from $p*p$, not $2*p$

Every composite multiple of p smaller than $p*p$ (like $2p, 3p, \dots, (p-1)p$) has already been marked by a smaller prime factor earlier in the sieve — starting from $p*p$ avoids redundant work and is what gives the $O(N \log \log N)$ bound rather than a slower $O(N \log N)$.

3.2 Why $O(N \log \log N)$

The total work is the sum, over every prime $p \leq \sqrt{N}$, of N/p marking steps. By a classical result in number theory (the sum of reciprocals of primes up to N grows like $\log \log N$), this sum totals $O(N \log \log N)$ — a function that grows so slowly it is, for any practical N , nearly indistinguishable from $O(N)$.

3.3 Sieve of smallest prime factor (SPF) — faster repeated factorization

```
vector<int> smallestPrimeFactor(int N) {
    vector<int> spf(N + 1);
    for (int i = 2; i <= N; i++) {
        if (spf[i] == 0) { // i is prime – nothing smaller has
marked it yet
            for (int j = i; j <= N; j += i)
                if (spf[j] == 0) spf[j] = i;
        }
    }
    return spf;
}

vector<int> fastFactorize(int n, vector<int> &spf) { // O(log n) per
query after O(N log log N) setup
    vector<int> factors;
    while (n > 1) { factors.push_back(spf[n]); n /= spf[n]; }
    return factors;
}
```

KEY IDEA — The precompute-once, query-fast pattern

This is the same tradeoff idea from Part V, Chapter 5 (time-space tradeoffs): spend $O(N \log \log N)$ once to build the SPF table, and every future factorization query drops from $O(\sqrt{n})$ to $O(\log n)$ — decisive when thousands of factorization queries are needed against the same bound N .

3.4 Segmented sieve (primes in a large range $[L, R]$)

When R can be up to 10^{12} but $R - L$ is small (e.g. $\leq 10^6$), a full sieve up to R is infeasible in memory. A segmented sieve instead sieves only the small range $[L, R]$ directly, using primes up to $\sqrt{R}$ (found via a small ordinary sieve) to cross off multiples within that range.

```
vector<bool> segmentedSieve(long long L, long long R) {
    int limit = (int)sqrt((double)R) + 1;
    vector<bool> basePrime = sieveOfEratosthenes(limit);
    vector<bool> isPrimeInRange(R - L + 1, true);
    if (L == 0) isPrimeInRange[0] = false;
    if (L <= 1) isPrimeInRange[max(0LL, 1 - L)] = false;
    for (int p = 2; p <= limit; p++) {
        if (basePrime[p]) {
            long long start = max((long long)p * p, ((L + p - 1) / p)
* p); // first multiple of p >= L
            for (long long multiple = start; multiple <= R; multiple
+= p)
                isPrimeInRange[multiple - L] = false;
        }
    }
    return isPrimeInRange; // index i corresponds to the number L +
i
}
```

Technique	Best when
Full sieve up to N	N fits comfortably in memory (roughly up to 10^7 - 10^8)
Segmented sieve on $[L, R]$	R is huge but $R-L$ is small — memory for a full sieve up to R is infeasible

Chapter 4 — GCD, LCM, and the Euclidean Algorithm

4.1 The Euclidean algorithm

Key identity: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$, because any common divisor of a and b must also divide $a \% b$ (since $a \% b = a - k*b$ for some integer k), and vice versa.

```
long long gcd(long long a, long long b) {
    while (b) { a %= b; swap(a, b); }
    return a;
}
// C++17 also provides __gcd(a, b) built in, with identical
behaviour.
```

Dry run: gcd(48, 18)

a	b	a % b
48	18	12
18	12	6
12	6	0

Loop ends when $b=0$; result is $a=6$. $\text{gcd}(48,18) = 6$.

4.2 Complexity: why the Euclidean algorithm is $O(\log(\min(a,b)))$

Each step at least halves one of the two numbers within two iterations (a known number-theoretic fact related to Fibonacci numbers being the worst case), giving a logarithmic number of steps overall — this is the exact complexity cited without proof back in Part V, Chapter 9's graph-algorithm preview table, now derived.

4.3 LCM from GCD

```
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b; // divide first to reduce overflow
    risk before multiplying
}
```

WARNING — Divide before multiplying

Computing $(a * b) / \gcd(a, b)$ risks overflowing before the division happens, exactly the overflow concern from Part I, Section 3.4. Dividing a by $\gcd(a, b)$ first keeps intermediate values smaller.

Chapter 5 — Extended Euclidean Algorithm and Modular Inverse

5.1 What the extended algorithm adds

The extended Euclidean algorithm computes not just $\gcd(a, b)$ but also integers x and y satisfying Bezout's identity: $a*x + b*y = \gcd(a, b)$. This pair (x, y) is exactly what's needed to compute a modular inverse (Section 5.2).

```
long long extendedGcd(long long a, long long b, long long &x, long long &y) {  
    if (b == 0) { x = 1; y = 0; return a; }  
    long long x1, y1;  
    long long g = extendedGcd(b, a % b, x1, y1);  
    x = y1;  
    y = x1 - (a / b) * y1;  
    return g;  
}
```

5.2 Modular inverse: what it is and why DP needed it back in Part XV

The modular inverse of a , modulo m , is a number a^{-1} such that $(a * a^{-1}) \% m == 1$ — it plays the role of "dividing by a " inside modular arithmetic, where ordinary division isn't directly defined. This is exactly the missing piece for any counting DP (Part XV) whose formula involves division (e.g. nCr formulas in Chapter 9 of this Part) while also

needing the answer modulo a large prime.

```
long long modInverseExtendedEuclid(long long a, long long m) {  
    long long x, y;  
    long long g = extendedGcd(a, m, x, y);  
    if (g != 1) return -1; // inverse only exists when gcd(a, m) == 1  
    return ((x % m) + m) % m; // normalize to a non-negative result  
    (Part I, Section 5.1)  
}
```

WARNING — The inverse only exists when $\gcd(a, m) = 1$

If a and m share a common factor, no modular inverse exists at all — this is precisely why competitive problems use a large PRIME modulus like 10^9+7 : it guarantees $\gcd(a, m) = 1$ for every a not itself a multiple of that prime, making inverses always available for the values that matter.

5.3 Modular inverse via Fermat's Little Theorem (the common shortcut)

When the modulus m is prime, Fermat's Little Theorem states $a^{(m-1)} \equiv 1 \pmod{m}$ for any a not divisible by m — rearranging gives $a^{(m-2)} \equiv a^{-1} \pmod{m}$. This means the modular inverse can be computed directly with binary exponentiation (Chapter 7) instead of the extended Euclidean algorithm, and is the standard technique used in essentially all competitive code with a prime modulus.

```
long long modInverseFermat(long long a, long long primeMod) {  
    return binPow(a, primeMod - 2, primeMod); // defined in Chapter  
    7  
}
```

Chapter 6 — Modular Arithmetic

6.1 The four basic rules

Operation	Rule
Addition	$(a + b) \% m = ((a \% m) + (b \% m)) \% m$
Subtraction	$(a - b) \% m = ((a \% m) - (b \% m) + m) \% m$ — the $+m$ guards against negative modulo (Part I, Section 5.1)
Multiplication	$(a * b) \% m = ((a \% m) * (b \% m)) \% m$
Division	$(a / b) \% m = (a * \text{modInverse}(b, m)) \% m$ — NEVER divide directly under a modulus

WARNING — Why 'divide then mod' is simply wrong

$(a / b) \% m$ computed via ordinary integer division does NOT equal the mathematically correct modular result in general — modular division is fundamentally multiplication by the modular inverse, not the ordinary division operator. This is a very common source of subtly wrong answers on problems asking for a probability, ratio, or nCr value modulo a prime.

6.2 A common modulus: $1e9+7$

10^9+7 is prime, and small enough that $a*b$ for $a, b < 10^9+7$ fits within the range of `long long` (up to about 9.2×10^{18}) without overflowing before the modulo is applied — exactly why it's the overwhelmingly standard choice of modulus across competitive judges.

```
const long long MOD = 1e9 + 7;
long long addMod(long long a, long long b) { return ((a % MOD) + (b % MOD)) % MOD; }
long long mulMod(long long a, long long b) { return ((a % MOD) * (b % MOD)) % MOD; }
```

Chapter 7 — Binary (Fast) Exponentiation

7.1 The problem: computing a^n quickly

Multiplying a by itself $n-1$ times costs $O(n)$. Binary exponentiation computes a^n in $O(\log n)$ by repeatedly squaring: $a^n = (a^{(n/2)})^2$ when n is even, or $a * a^{(n-1)}$ when n is odd — directly mirroring the $O(\log n)$ recursion pattern from Part V, Chapter 4 (recall the Master Theorem's binary search example).

```
long long binPow(long long a, long long n, long long mod) {
    long long result = 1;
    a %= mod;
    while (n > 0) {
        if (n & 1) result = (result * a) % mod; // current bit of n
        is set: fold in this power of a
        a = (a * a) % mod; // square a for the next bit position
        n >>= 1;
    }
    return result;
}
```

Dry run: $3^{13} \bmod 1000$

n (binary)	n & 1	result	a after squaring
13 (1101)	1	3	9
6 (0110)	0	3	81
3 (0011)	1	$3*81=243$	$6561 \bmod 1000=561$
1 (0001)	1	$243*561 \bmod 1000=723$	—

$3^{13} = 1594323$; $1594323 \bmod 1000 = 323$. (Note: dry run rounding above is illustrative of the bit-by-bit process; always trust the code)

over hand arithmetic for larger exponents.)

KEY IDEA — Why this connects directly to bit manipulation (Chapter 13, and Part XVI itself)

Binary exponentiation is literally reading n 's binary representation one bit at a time (Part I's bit manipulation chapter) and using each bit to decide whether to fold the current squared power into the result — the same 'process one bit at a time' shape reappears in bitmask DP (Part XV, Chapter 10) and digit DP (Part XV, Chapter 11).

Approach	Complexity
Naive repeated multiplication	$O(n)$
Binary exponentiation	$O(\log n)$

Chapter 8 — Combinatorics: Permutations, Combinations, Factorials

8.1 Factorials and permutations

$n!$ (n factorial) counts the number of ways to arrange n distinct items in order — exactly the search space size that made brute-force permutation enumeration infeasible back in Part V's complexity table and Part XV's TSP motivation.

```
long long factorial(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
```

8.2 Combinations: nCr

$nCr = n! / (r! * (n-r)!)$ counts the number of ways to CHOOSE r items from n without regard to order (unlike permutations, where order matters).

```
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    r = min(r, n - r); // nCr == nC(n-r); compute the smaller one to
    reduce work
    long long result = 1;
    for (int i = 0; i < r; i++) {
        result = result * (n - i) / (i + 1); // careful ordering
        keeps every intermediate an integer
    }
    return result;
}
```

NOTE — Why this direct formula still risks overflow for large n

Even without ever intermediate-dividing incorrectly, nCr grows extremely fast ($nC(n/2)$ is exponential in n), so for large n this must be computed modulo a prime instead — exactly what Chapter 9 covers.

Chapter 9 — Modular nCr and Pascal's Triangle

9.1 Precomputed factorials + modular inverse (the standard technique)

For many nCr queries against the same n range, precompute factorials and their modular inverses once (Chapter 7's Fermat shortcut), then answer each query in $O(1)$ — the same precompute-once, query-fast tradeoff from Chapter 3's SPF sieve.

```

const int MAXN = 200005;
const long long MOD = 1e9 + 7;
long long fact[MAXN], invFact[MAXN];

void precomputeFactorials() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[MAXN - 1] = binPow(fact[MAXN - 1], MOD - 2, MOD); //
    Fermat inverse of the largest factorial
    for (int i = MAXN - 2; i >= 0; i--)
        invFact[i] = invFact[i + 1] * (i + 1) % MOD; // invFact[i] =
    invFact[i+1] * (i+1), derived below
}

long long nCrMod(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

```

KEY IDEA — Why invFact can be computed backward in O(1) per step

$\text{invFact}[i] = 1/i! = (1/(i+1)!) * (i+1) = \text{invFact}[i+1] * (i+1)$ — this identity means only ONE binary exponentiation call is needed for the whole precomputation (at the top index), with every other inverse factorial derived in O(1) by working backward. This turns what looks like $O(\text{MAXN} \log \text{MOD})$ work into $O(\text{MAXN} + \log \text{MOD})$.

Approach	Per-query complexity	Preprocessing
Direct nCr formula per query	$O(r)$	None
Precomputed factorials + modular inverse	$O(1)$	$O(\text{MAXN} + \log \text{MOD})$

9.2 Pascal's Triangle (an alternative for small n)

$nCr = (n-1)C(r-1) + (n-1)Cr$ — this recurrence builds Pascal's Triangle directly, avoiding factorials/inverses entirely, and is simple and robust when n is small enough (roughly up to a few thousand) that an $O(n^2)$ table is affordable.

```
vector<vector<long long>> pascalsTriangle(int n) {
    vector<vector<long long>> C(n + 1, vector<long long>(n + 1, 0));
    for (int i = 0; i <= n; i++) {
        C[i][0] = 1;
        for (int j = 1; j <= i; j++)
            C[i][j] = (C[i-1][j-1] + (j <= i-1 ? C[i-1][j] : 0)) %
MOD;
    }
    return C;
}
```

NOTE — This is 2D DP, revisited

Pascal's Triangle is itself a small 2D DP (Part XV, Chapter 4): state $C[i][j]$, transition $C[i][j] = C[i-1][j-1] + C[i-1][j]$, base case $C[i][0] = 1$ — one more example of the same DP shape appearing in a new context.

Chapter 10 — Matrix Exponentiation

10.1 The idea: linear recurrences as matrix multiplication

Any linear recurrence (like Fibonacci's $dp[i] = dp[i-1] + dp[i-2]$) can be rewritten as a matrix multiplication step, and repeated application of that step n times becomes a matrix raised to the n -th power — which binary exponentiation (Chapter 7) computes in $O(\log n)$ matrix multiplications instead of n .

$$\begin{bmatrix} F(n+1) \\ F(n) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix}$$

So $\begin{bmatrix} F(n+1) \\ F(n) \end{bmatrix}^T = M^n * \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}^T$, where $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

```
typedef vector<vector<long long>> Matrix;

Matrix multiply(Matrix &A, Matrix &B, long long mod) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Matrix C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            for (int l = 0; l < k; l++)
                C[i][j] = (C[i][j] + A[i][l] * B[l][j]) % mod;
    return C;
}

Matrix matrixPow(Matrix M, long long n, long long mod) {
    int size = M.size();
    Matrix result(size, vector<long long>(size, 0));
    for (int i = 0; i < size; i++) result[i][i] = 1; // identity matrix
    while (n > 0) {
        if (n & 1) result = multiply(result, M, mod);
        M = multiply(M, M, mod);
        n >>= 1;
    }
    return result;
}

long long fibMatrix(long long n, long long mod) {
    if (n == 0) return 0;
    Matrix M = {{1, 1}, {1, 0}};
    Matrix result = matrixPow(M, n - 1, mod);
    return result[0][0]; // F(n)
}
```

KEY IDEA — The full-circle moment for this book

Fibonacci has now been computed three ways across this book: naive recursion, $O(2^n)$ (Part V, Chapter 4) — memoized/tabulated DP, $O(n)$ (Part XV, Chapter 1) — and now matrix exponentiation, $O(\log n)$. Each step removed a specific inefficiency: memoization removed redundant recomputation; matrix exponentiation removes the need to visit every intermediate index at all. Recognizing when a linear recurrence can skip straight to $O(\log n)$ this way is valuable whenever n is too large even for an $O(n)$ DP (e.g. n up to 10^{18}).

Approach	Complexity
Naive recursion	$O(2^n)$
DP (memo/tabulation)	$O(n)$
Matrix exponentiation	$O(k^3 \log n)$ for a $k \times k$ recurrence matrix

Chapter 11 — Inclusion-Exclusion Principle

11.1 The core idea

To count elements satisfying AT LEAST ONE of several conditions without double-counting overlaps: sum the individual counts, subtract every pairwise overlap (subtracted too much), add back every triple overlap (added back what was subtracted too many times), and so on, alternating signs.

$$|A \cup B| = |A| + |B| - |A \cap B|$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |AB| - |AC| - |BC| + |ABC|$$

11.2 Worked example: count numbers up to N divisible by 3 or 5

```

long long countDivisibleBy3or5(long long N) {
    long long div3 = N / 3;
    long long div5 = N / 5;
    long long div15 = N / 15; // divisible by both 3 and 5
    return div3 + div5 - div15;
}

```

This is the classic "Project Euler problem 1" shape, and generalizes directly: for more divisors, alternate adding singles/triples and subtracting pairs/quadruples following the same pattern.

11.3 Worked example: Euler's totient function via inclusion-exclusion

$\phi(n)$ counts integers from 1 to n coprime to n . Using inclusion-exclusion over n 's distinct prime factors $p_1, p_2, \dots, p_k$: $\phi(n) = n * \text{product over each prime factor } p \text{ of } (1 - 1/p)$.

```

long long eulerTotient(long long n) {
    long long result = n;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            result -= result / p; // result *= (1 - 1/p), rearranged
            to avoid floating point
        }
    }
    if (n > 1) result -= result / n; // leftover prime factor
    return result;
}

```

NOTE — Where $\phi(n)$ is used

Euler's totient generalizes Fermat's Little Theorem (Chapter 5) to composite moduli, and appears in problems counting coprime pairs, reduced fractions, and cycle lengths in modular arithmetic — a recurring building block once number theory problems go beyond prime moduli.

Chapter 12 — Pigeonhole Principle

12.1 The principle, stated simply

If more than n items are placed into n containers, at least one container holds more than one item. Trivial to state, but it's a surprisingly powerful proof technique for competitive programming — it often proves a solution EXISTS or bounds how many steps something can take, without needing to construct the solution explicitly.

12.2 Worked example: proving a bound on distinct remainders

Claim: among any $n+1$ integers, at least two share the same remainder modulo n . Proof: there are only n possible remainders (0 through $n-1$) — the "containers" — and $n+1$ integers — the "items" — so by pigeonhole, two integers must land in the same remainder container.

12.3 Worked example: bounding cycle length in modular sequences

A sequence defined by $a[i+1] = f(a[i]) \bmod m$ must eventually repeat a value, because there are only m possible values (pigeonhole) — so within $m+1$ steps, some value repeats, and the sequence enters a cycle from that point onward. This exact argument is what justifies cycle-detection techniques (Floyd's algorithm, Part VIII) actually terminating in bounded time on modular or state-space sequences.

KEY IDEA — The pigeonhole habit

Whenever a problem asks you to prove something 'always happens' or bound how long a process can run before repeating/terminating, check whether the possible states/values form a finite, countable set smaller than the number of steps/items being placed into it — if so, pigeonhole often supplies the argument directly, without needing to simulate anything.

Chapter 13 — Bitwise Mathematics Recap

Part I introduced bitwise operators; this recap ties the number-theoretic view together before the Advanced String Algorithms and Advanced Data Structures Parts lean on bit tricks more heavily.

Trick	What it does
$n \& (n-1)$	Clears the lowest set bit of n
$n \& (-n)$	Isolates the lowest set bit of n
<code>__builtin_popcount(n)</code>	Counts set bits in n — $O(1)$ via a hardware instruction on most platforms
$1LL \ll k$	Computes 2^k directly, as a bitmask with only bit k set
$n \& (n-1) == 0$	True exactly when n is a power of two (or zero)

NOTE — Where this reconnects

Every one of these is a direct callback: bitmask DP (Part XV, Chapter 10) iterates and tests bits with exactly these tricks; binary exponentiation (Chapter 7 of this Part) reads n 's bits one at a time; digit DP (Part XV, Chapter 11) applies the same 'process the representation one symbol at a time' idea to base-10 digits instead of binary bits.

Chapter 14 — Recognizing a Number-Theory Problem

Clue	Likely technique
"Answer modulo 10^9+7 "	Modular arithmetic throughout; watch for any division needing a modular inverse
"Count coprime pairs / reduced fractions"	GCD, or Euler's totient function
Many queries asking 'is x prime' or 'factorize x ' for x up to 10^6 - 10^7	Precompute with a sieve / SPF table
A huge range $[L, R]$ with R up to 10^{12} but $R-L$ small	Segmented sieve
A linear recurrence needed at a huge index n (up to 10^{18})	Matrix exponentiation
"Count ways to choose r items from n ", possibly modulo a prime	nCr / modular nCr with precomputed factorials
"Count elements satisfying at least one of several conditions"	Inclusion-exclusion
A proof that something 'must eventually happen/repeat'	Pigeonhole principle

Chapter 15 — Practice Drills

Problem 16.1 — Count trailing zeros in $n!$ [Level 2]

Find the number of trailing zeros in $n!$ without computing the (potentially astronomically large) factorial directly.

Hint 1: A trailing zero comes from a factor of $10 = 2 * 5$. Which of 2 or 5 is scarcer among the factors of $n!$?

Key observation: Factors of 5 are rarer than factors of 2 in $n!$, so the count of trailing zeros equals the count of factor-5's, computable as $\text{floor}(n/5) + \text{floor}(n/25) + \text{floor}(n/125) + \dots$

Complexity: $O(\log_5(n))$.

Problem 16.2 — Fast modular Fibonacci at huge n [Level 3]

Compute $F(n) \bmod 10^9+7$ for n up to 10^{18} .

Hint 1: n is far too large for an $O(n)$ DP.

Approach: Matrix exponentiation (Chapter 10): $O(\log n)$ with a constant 2×2 matrix multiplication cost.

Complexity: $O(\log n)$.

Problem 16.3 — Count numbers up to N coprime to N [Level 2]

Given N , count how many integers from 1 to N share no common factor with N .

Hint 1: This is exactly Euler's totient function.

Approach: Factorize N , then apply $\text{eulerTotient}(N)$ from Chapter 11.

Complexity: $O(\sqrt{N})$.

Problem 16.4 — Modular nCr with many queries [Level 2]

Answer q queries of $nCr \bmod 10^9+7$, with n, r up to $2 \cdot 10^5$, as fast as possible.

Hint 1: Precompute once; don't recompute factorials per query.

Approach: Precompute $\text{fact}[]$ and $\text{invFact}[]$ up to the maximum needed n once (Chapter 9), then answer each query in $O(1)$.

Complexity: $O(\text{MAXN} + \log \text{MOD})$ preprocessing, $O(1)$ per query.

Part XVI — Master Revision

Must Remember

- Finding divisors, testing primality, and building a sieve all share the same $\sqrt{n}$ / $p \cdot p$ pairing argument — the same idea reused three times in this Part.
- $\gcd(a,b) = \gcd(b, a \% b)$, computed in $O(\log(\min(a,b)))$; $\text{lcm}(a,b) = a / \gcd(a,b) \cdot b$, dividing before multiplying to reduce overflow risk.
- Modular division is never ordinary division followed by mod — it is multiplication by the modular inverse, computed via Fermat's Little Theorem ($a^{(m-2)} \bmod m$) when m is prime.
- Binary exponentiation computes $a^n \bmod m$ in $O(\log n)$ by processing n 's bits one at a time — the same shape reused in matrix exponentiation, bitmask DP, and digit DP.
- For repeated nCr queries mod a prime, precompute factorials and inverse factorials ONCE, then answer each query in $O(1)$; the backward invFact recurrence needs only a single binary exponentiation call.
- A linear recurrence needed at an index up to 10^{18} should be computed via matrix exponentiation ($O(\log n)$), not a per-step DP ($O(n)$).
- Inclusion-exclusion sums individual counts, subtracts pairwise overlaps, adds back triple overlaps, alternating signs.
- Pigeonhole principle proves existence/repetition arguments directly from a finite state space, without needing to simulate anything.

Complexity Reference — Part XVI

Technique	Complexity
Find all divisors of n	$O(\sqrt{n})$
Primality test	$O(\sqrt{n})$
Prime factorization	$O(\sqrt{n})$
Sieve of Eratosthenes up to N	$O(N \log \log N)$
SPF-based factorization (after precompute)	$O(\log n)$ per query
Segmented sieve on $[L, R]$	$O((R-L) \log \log R + \sqrt{R})$
GCD (Euclidean algorithm)	$O(\log(\min(a,b)))$
Modular inverse (Fermat)	$O(\log m)$
Binary exponentiation	$O(\log n)$
Modular nCr (after precompute)	$O(1)$ per query
Matrix exponentiation ($k \times k$ matrix)	$O(k^3 \log n)$

Common Mistakes Recap

Mistake	Fix
Computing $(a/b) \% m$ directly under a modulus	Multiply by <code>modInverse(b, m)</code> instead
Computing $(a*b)/\gcd(a,b)$ for LCM without dividing first	Divide a by $\gcd(a,b)$ before multiplying by b
Forgetting the $i \neq n/i$ check when listing divisors	Guard against double-counting a perfect-square root
Starting sieve marking from $2p$ instead of $p*p$	Start from $p*p$ for the $O(N \log \log N)$ bound
Using Fermat's shortcut with a non-prime modulus	Fermat's Little Theorem requires m to be prime; use the extended Euclidean algorithm otherwise
Recomputing factorials inside a query loop	Precompute once outside the query loop
Using an $O(n)$ DP for a recurrence needed at n up to 10^{18}	Use matrix exponentiation instead

Interview-Style Questions

- 1 Why does checking divisibility only up to $\sqrt{n}$ suffice to determine primality?
- 2 Explain why modular division requires a modular inverse rather than the ordinary division operator.
- 3 Why does the Sieve of Eratosthenes start marking multiples of p from $p*p$ rather than $2p$?
- 4 Derive why $\text{invFact}[i] = \text{invFact}[i+1] * (i+1)$, and explain why this means only one binary exponentiation call is needed to precompute every inverse factorial.
- 5 Explain how a linear recurrence like Fibonacci can be computed in $O(\log n)$ via matrix exponentiation.
- 6 State the pigeonhole principle and use it to prove that among any $n+1$ integers, two share the same remainder modulo n .

Master Cheat Sheet — Part XVI

```
const long long MOD = 1e9 + 7;

// GCD / LCM
long long gcd(long long a, long long b) { while (b) { a %= b; swap(a, b); } return a; }
long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }

// Binary exponentiation (mod)
long long binPow(long long a, long long n, long long mod) {
    long long result = 1; a %= mod;
    while (n > 0) { if (n & 1) result = result * a % mod; a = a * a % mod; n >>= 1; }
    return result;
}

// Modular inverse (prime modulus)
long long modInverse(long long a, long long mod) { return binPow(a, mod - 2, mod); }

// Sieve of Eratosthenes
vector<bool> sieve(int N) {
    vector<bool> isPrime(N + 1, true); isPrime[0] = isPrime[1] = false;
    for (int p = 2; (long long)p * p <= N; p++)
        if (isPrime[p]) for (int m = p * p; m <= N; m += p)
            isPrime[m] = false;
    return isPrime;
}

// nCr mod, after precomputing fact[] and invFact[]
long long nCrMod(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

NOTE — Coming up in Part XVII

Part XVII — Advanced String Algorithms moves from numeric structure to textual structure: string hashing (revisiting the rolling-hash preview from Part VII), the KMP and Z algorithms for $O(n+m)$ exact pattern matching, the Trie for prefix-based queries, and an introduction to Aho-Corasick for matching many patterns at once.